

طراحی و پیاده سازی زبانهای برنامه سازی

فصل اول: اصول و طراحی سیستمها

فصل دوم: اثرات معماری ماشین

فصل سوم: اصول ترجمه زبان

فصل چهارم: عناصر نحوی زبان

فصل پنجم: انواع داده اولیه

فصل ششم: بسته بندی

فصل هفتم: کنترل ترتیب اجرا

فصل هشتم: کنترل ترتیب زیر برنامه

فصل اول

اصول طراحی زبانها

علت مطالعه زبانهای مختلف

- برای بهبود توانایی خود در توسعه الگوریتمهای کارآمد
- استفاده بهینه از زبان برنامه نویسی موجود
- می توانید با اصلاحات مفید ساختارهای برنامه نویسی آشنا شوید.
- انتخاب بهترین زبان برنامه سازی
- آموزش زبان جدید ساده می شود.
- طراحی زبان جدید ساده می شود.

تاریخچه مختصری از زبانهای برنامه سازی

YEAR	LANGUAGE
1950-55	Assembly
1956-60	Fortran , Algol 58 , Algol 60 , Cobol , Lisp
1961-65	FortranIR , Cobol 61 , Algol 61 , Snobol , APL
1966-70	Fortran 66 , Cobol 65 , Snobol 4 , Simula 67 , Basic , APL 360
1971-75	Pascal , Cobol 74 , APL (standard) , C
1976-80	Ada , Fortran 77 (standard)
1981-85	Prolog
1985-90	C++ , Fortran 90

تاریخچه مختصری از زبانهای برنامه سازی

توسعه زبانهای اولیه (ادامه)

• زبانهای تجاری (1955)

- تسريع در ورود اطلاعات ، نگهداری خروجی و سابقه در قالب جدول و نمودار
- زبان کوبول در ارتش آمریکا

• زبان هوش مصنوعی (دهه 1950)

- لیست به عنوان زبان پردازش کننده لیست بوجود آمد
- پیاده سازی بازیهای کامپیوتری
- عملیات جستجو
- امکانات پردازش متن
- رشته ای از نمادها با رشته دیگر از نمادها جایگزین شود.

• زبانهای سیستم

- برای نوشتن نرم افزارهای سیستمی مانند سیستم عامل
- پیاده سازی کامپایلر

تاریخچه مختصری از زبانهای برنامه سازی

توسعه زبانهای اولیه (ادامه)

• زبانهای محاسباتی

◦ فرترن و الگول

- محاسبات با حجم بالا
- جهت توصیف الگوریتم ها
- نباید وابسته به ماشین خاصی باشد

تاریخچه مختصری از زبانهای برنامه سازی (ادامه)

2. تکامل معماری نرم افزار

دوران کامپیوترهای بزرگ

- محیط دسته ای (فرترن ، کوبول و پاسکال در ابتدا طراحی شدند)
 - کاربر با برنامه هیچ مراوده ای ندارد
 - ورودیها ، مجموعه ای از فایلها هستند
 - اصلاح خطا بسیار هزینه بر است
 - محدوده زمانی مشخصی ندارد

● محیط محاوره ای (C و C++)

- کاربر مستقیم با برنامه ارتباط دارد
- خروجی را روی صفحه میبیند
- برای انجام کارها از سیستم اشتراک زمانی استفاده میشود.
- تاثیر بر طراحی زبان

تاریخچه مختصری از زبانهای برنامه سازی (ادامه)

2. تکامل معماری نرم افزار

- کامپیوترهای شخصی
 - دوران کامپیوتر شخصی (C++ و جاوا)
 - اهمیت کم کارایی
 - داشتن گرافیک ساده
- محیطهای سیستم تعبیه شده (Ada, C, C++)
 - جهت کنترل بخشی از هواپیما یا ماشین آلات صنعتی
 - اهمیت بالای خرابی
 - به صورت بلادرنگ هستند
 - اهمیت بالای قابلیت اطمینان
 - بی نیازی به سیستم عامل ، سیستم فایل و دستگاههای I/O
 - تاثیر بر طراحی زبان



تاریخچه مختصری از زبانهای برنامه سازی (ادامه)

تکامل معماری نرم افزار (ادامه)

دوران شبکه بندی

- محاسبات توزیعی
- اینترنت
- تاثیر بر زبان برنامه سازی

تاریخچه مختصری از زبانهای برنامه سازی (ادامه)

3. دامنه های کاربرد

کاربردها در دهه 1960

- پردازش تجاری
- محاسبات علمی
- برنامه نویسی سیستم
- کاربردهای هوش مصنوعی

تاریخچه مختصری از زبانهای برنامه سازی (ادامه)

دامنه های کاربرد (ادامه)

کاربردهای قرن 21

- پردازش تجاری
- محاسبات علمی
- برنامه نویسی سیستم
- کاربردهای هوش مصنوعی
- انتشارات
- کاربردهای جدید (مانند شی گراها و...)؛ مانند کاربرد ام ال در تحقیقات زبانهای برنامه سازی

دامنه های کاربردزبانها

دوره	کاربرد	زبان مورد استفاده
۱۹۶۰	تجاری	Cobol
	علمی	Fortran,Basic,Algol,APL
	سیستمی	Assembly
	هوش مصنوعی	Lisp,Snobol
امروزه	تجاری	C++,Java,4GL
	علمی	Java,C,C++,Basic
	سیستمی	C,C++,Java
	هوش مصنوعی	Lisp,Prolog
	انتشارات	Tex,Postscript

نقش زبانهای برنامه سازی

تغییرات بوجود آمده و اثرات آنها بر زبانهای برنامه سازی

- تغییر در قابلیت‌های کامپیوتر (کامپیوترهای بزرگ ، کند و گرانقیمت که از لامپ خلا استفاده می کردند به ریز کامپیوترها و سوپر کامپیوترها تبدیل شدند) : ساختار و هزینه های استفاده از زبانهای سطح بالا تحت تاثیر قرار گرفت.
- زمینه های کاربرد جدید: موجب طراحی زبانهای جدید ، ارتقاء و بازبینی زبانهای قدیمی
- یافتن متدهای برنامه نویسی خوب برای برنامه های بزرگ و پیچیده و تغییر در محیط های برنامه نویسی: موجب رشد در طراحی زبان ها شد.
- متدهای پیاده سازی : انتخاب ویژگیهای نو برای طراحی های جدید
- مطالعات تئوری: استفاده از متدهای رسمی ریاضیات
- نیاز به انتقال برنامه از کامپیوتری به کامپیوتر دیگر: موجب استانداردسازی در زبانها

نقش زبانهای برنامه سازی (ادامه)

زبان خوب چگونه است ؟

صفات یک زبان خوب

- وضوح ، سادگی و یکپارچگی :

جامعیت مفهومی : مفاهیم و ابزارهای موجود در یک زبان و قوانین ترکیب آنها در یک زبان برنامه سازی خوانایی برنامه : تفاوت های معنایی منعکس کننده تفاوت های نحوی باشد.

- قابلیت تعامل : امکان ترکیب ویژگی های مختلف زبان و با معنا بودن ترکیب حاصل

مثال : ترکیب عبارت و ساختار شرطی

مزیت : یادگیری زبان ساده و نوشتن برنامه راحت

معایب : کامپایل بدون خطا در ترکیب هایی که منطق روشن و اجرای کارآمدی ندارند.

- طبیعی بودن برای کاربردها

زبانها باید ساختمان داده ، عملگرها ، دستورات کنترلی و نحو مناسب برای مسئله ای که باید حل شود را داشته باشند. برای هر کاربرد خاص یک زبان مناسب آن

نقش زبانهای برنامه سازی (ادامه)

1. صفات یک زبان خوب (ادامه)

- پشتیبانی از انتزاع
- سهولت در بازرسی برنامه : درستی صحت و عملکرد برنامه
 - (بازرسی رسمی (ریاضی) و غیر رسمی (تست برنامه با ورودیهای مختلف))
- محیط برنامه نویسی : وجود ویراستارهای خاص ، امکانات نگهداری و اصلاح نسخه های متفاوت
- قابلیت حمل برنامه : وابستگی نداشتن به زبان خاص
- هزینه استفاده
 - هزینه اجرای برنامه : منظور استفاده از `cpu, hard disk` میباشد. بستگی به کامپایلر دارد ولی امروزه زیاد مهم نیست.
 - هزینه ترجمه برنامه : هزینه کامپایل کردن. در برنامه های دانشجویی برنامه به تعداد زیاد ترجمه میشود تا اجرا
 - هزینه نگهداری برنامه : هزینه های ترمیم خطا بعد از اجرا ، توسعه و تغییر سیستم عامل و ..

نقش زبانهای برنامه سازی (ادامه)

نحو و معنای زبان

- نحو (syntax) زبان برنامه سازی ، ظاهر آن زبان است.
- قواعد نحوی مشخص می کنند که دستورات ، اعلانها و سایر ساختارهای زبان چگونه نوشته می شوند
- معنای (semantic) زبان همان مفهومی است که به ساختارهای نحوی زبان داده می شود.
- `v: array [0..9] of integer;`
- `int v[10];`

1-3- نقش زبانهای برنامه سازی (ادامه)

2. مدل‌های زبان (محاسباتی ، تجاری ، هوش مصنوعی و سیستمی)

محاسباتی:

- زبانهای دستوری (imperative): زبانهای مبتنی بر فرمان یا دستورگرا
مانند C++ , C و پاسکال و ...
- زبانهای تابعی (applicative): به جای مشاهده تغییر حالت ماشین ، عملکرد برنامه دنبال می شود. توابع در کنار هم برنامه را تشکیل میدهند و قابلیت فراخوانی تودرتو را دارند.
مانند ام ال و لیسپ (بعضی وقتها C)
- $$\text{function}_n(\dots(\text{function}_2(\text{function}_1(\text{data})) \dots)$$
- زبانهای قانونمند (rule-based): شرایطی را بررسی می کنند و در صورت برقرار بودن آنها فعالیت را انجام می دهند.
$$\text{enable condition}_1 \longrightarrow \text{action}_1$$
مانند پرولوگ
- برنامه نویسی شی گرا (object-oriented): اشیای پیچیده به عنوان بسطی از اشیای ساده ساخته می شوند و خواصی را از اشیای ساده به ارث می برند. شامل ماژول ها هستند که در آن تعریف داده ها و عملیات آنها وجود دارد.

نقش زبانهای برنامه سازی (ادامه)

3. استاندارد سازی زبان

روش پی بردن به معنای دستورات و خروجی :

- به مستندات زبان مراجعه شود.
- برنامه را در کامپیوتر تایپ و اجرا کنید
- به استاندارد زبان مراجعه شود.

استانداردهای زبان دو دسته اند :

- استاندارد خصوصی : توسط شرکت یا مالک زبان ارائه می شوند.
- استاندارد عمومی : اسنادی که توسط سازمانهای معروف به توافق رسیده اند.

مسائل مهم در استفاده ی موثر از استاندارد:

- زمان سنجی : چه زمانی باید زبان استاندارد شود ؟
- اطاعت و پیروی : برنامه نویس باید مراقب ویژگیهای اضافی که در کامپایلر وجود دارد باشد. کامپایلر برنامه های مطابق با استاندارد زبان را ترجمه میکند.
- کهنگی : کی استاندارد کهنه می شود و چگونه باید آن را اصلاح کرد ؟

محیط های برنامه نویسی

1. محیط برنامه نویسی در دو زمینه بر طراحی زبان تاثیر گذاشته است :

- کامپایل کردن مجزای زیربرنامه و سایر بخشهای برنامه

- کامپایلر باید این اطلاعات را داشته باشد:

مشخه ی تعداد ، ترتیب و نوع پارامترهای زیربرنامه

اعلان نوع داده

تعریف نوع داده

- تست و اشکال زدایی

- مانند : ویژگیهای ردیابی اجرا ، نقاط کنترلی

محیط های برنامه نویسی (ادامه)

2. محیط های کاری

- محیط کاری ، خدماتی مثل ذخیره داده ها ، رابط گرافیکی کاربر ، امنیت و خدمات ارتباطی را فراهم می کند.

محیط های برنامه نویسی (ادامه)

3. زبانهای کنترل کار و فرآیند

- مفهوم کنترل کار به چارچوبهای محیط برمی گردد.
- کاربر کنترل مستقیم بر روی مراحل مختلف برنامه دارد.